JavaScript is disabled on your browser.

compact1, compact2, compact3

java.io

Class Writer

java.lang.Object
    java.io.Writer

All Implemented Interfaces:

`Closeable`, `Flushable`, `Appendable`, `AutoCloseable`

Direct Known Subclasses:

`BufferedWriter`, `CharArrayWriter`, `FilterWriter`, `OutputStreamWriter`, `PipedWriter`, `PrintWriter`, `StringWriter`

```
public abstract class Writer
extends Object
implements Appendable, Closeable, Flushable
```

Abstract class for writing to character streams. The only methods that a subclass must implement are write(char[], int, int), flush(), and close(). Most subclasses, however, will override some of the methods defined here in order to provide higher efficiency, additional functionality, or both.

Since:

JDK1.1

See Also:

`Writer`, `BufferedWriter`, `CharArrayWriter`, `FilterWriter`, `OutputStreamWriter`, `FileWriter`, `PipedWriter`, `PrintWriter`, `StringWriter`, `Reader`

Field Summary

Fields

Modifier and Type	Field	Description

protected **Object**	**lock**	The object used to synchronize operations on this stream.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**Writer**()	Creates a new character-stream writer whose critical sections will synchronize on the writer itself.
protected	**Writer**(**Object** lock)	Creates a new character-stream writer whose critical sections will synchronize on the given object.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
Writer	**append**(char c)	Appends the specified character to this writer.
Writer	**append**(**CharSequence** csq)	Appends the specified character sequence to this writer.
Writer	**append**(**CharSequence** csq, int start, int end)	Appends a subsequence of the specified character sequence to this writer.
abstract void	**close**()	Closes the stream, flushing it first.
abstract void	**flush**()	Flushes the stream.
void	**write**(char[] cbuf)	Writes an array of characters.
abstract void		

	write(char[] cbuf, int off, int len)	Writes a portion of an array of characters.
void	**write**(int c)	Writes a single character.
void	**write**(**String** str)	Writes a string.
void	**write**(**String** str, int off, int len)	Writes a portion of a string.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

lock

```
protected Object lock
```

The object used to synchronize operations on this stream. For efficiency, a character-stream object may use an object other than itself to protect critical sections. A subclass should therefore use the object in this field rather than `this` or a synchronized method.

Constructor Detail

Writer

```
protected Writer()
```

Creates a new character-stream writer whose critical sections will synchronize on the writer itself.

Writer

```
protected Writer(Object lock)
```

Creates a new character-stream writer whose critical sections will synchronize on the given object.

Parameters:

lock - Object to synchronize on

Method Detail

write

```
public void write(int c)
           throws IOException
```

Writes a single character. The character to be written is contained in the 16 low-order bits of the given integer value; the 16 high-order bits are ignored.

Subclasses that intend to support efficient single-character output should override this method.

Parameters:

c - int specifying a character to be written

Throws:

IOException - If an I/O error occurs

write

```
public void write(char[] cbuf)
           throws IOException
```

Writes an array of characters.

Parameters:

cbuf - Array of characters to be written

Throws:

IOException - If an I/O error occurs

write

```
public abstract void write(char[] cbuf,
                           int off,
                           int len)
                    throws IOException
```

Writes a portion of an array of characters.

Parameters:

cbuf - Array of characters

off - Offset from which to start writing characters

len - Number of characters to write

Throws:

IOException - If an I/O error occurs

write

```
public void write(String str)
           throws IOException
```

Writes a string.

Parameters:

str - String to be written

Throws:

IOException - If an I/O error occurs

write

```
public void write(String str,
                  int off,
                  int len)
           throws IOException
```

Writes a portion of a string.

Parameters:

str - A String

off - Offset from which to start writing characters

len - Number of characters to write

Throws:

IndexOutOfBoundsException - If off is negative, or len is negative, or off+len is negative or greater than the length of the given string

IOException - If an I/O error occurs

append

```
public Writer append(CharSequence csq)
              throws IOException
```

Appends the specified character sequence to this writer.

An invocation of this method of the form `out.append(csq)` behaves in exactly the same way as the invocation

```
out.write(csq.toString())
```

Depending on the specification of `toString` for the character sequence `csq`, the entire sequence may not be appended. For instance, invoking the `toString` method of a character buffer will return a subsequence whose content depends upon the buffer's position and limit.

Specified by:

`append` in interface `Appendable`

Parameters:

`csq` - The character sequence to append. If csq is null, then the four characters "null" are appended to this writer.

Returns:

This writer

Throws:

`IOException` - If an I/O error occurs

Since:

1.5

append

```
public Writer append(CharSequence csq,
                     int start,
                     int end)
              throws IOException
```

Appends a subsequence of the specified character sequence to this writer. `Appendable`.

An invocation of this method of the form `out.append(csq, start, end)` when `csq` is not `null` behaves in exactly the same way as the invocation

```
out.write(csq.subSequence(start, end).toString())
```

Specified by:

`append` in interface `Appendable`

Parameters:

`csq` - The character sequence from which a subsequence will be appended. If csq is null, then characters will be appended as if csq contained the four characters "null".

`start` - The index of the first character in the subsequence

end - The index of the character following the last character in the subsequence

Returns:

This writer

Throws:

IndexOutOfBoundsException - If start or end are negative, start is greater than end, or end is greater than csq.length()

IOException - If an I/O error occurs

Since:

1.5

append

```
public Writer append(char c)
              throws IOException
```

Appends the specified character to this writer.

An invocation of this method of the form `out.append(c)` behaves in exactly the same way as the invocation

```
out.write(c)
```

Specified by:

append in interface Appendable

Parameters:

c - The 16-bit character to append

Returns:

This writer

Throws:

IOException - If an I/O error occurs

Since:

1.5

flush

```
public abstract void flush()
                    throws IOException
```

Flushes the stream. If the stream has saved any characters from the various write() methods in a buffer, write them immediately to their intended destination. Then, if that destination is another character or byte stream,

flush it. Thus one flush() invocation will flush all the buffers in a chain of Writers and OutputStreams.

If the intended destination of this stream is an abstraction provided by the underlying operating system, for example a file, then flushing the stream guarantees only that bytes previously written to the stream are passed to the operating system for writing; it does not guarantee that they are actually written to a physical device such as a disk drive.

Specified by:

`flush` in interface `Flushable`

Throws:

`IOException` - If an I/O error occurs

close

```
public abstract void close()
                    throws IOException
```

Closes the stream, flushing it first. Once the stream has been closed, further write() or flush() invocations will cause an IOException to be thrown. Closing a previously closed stream has no effect.

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `AutoCloseable`

Throws:

`IOException` - If an I/O error occurs

workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.